100 Java Viva Questions with Answers

Student-friendly revision guide • Six sections • Viva-ready explanations

A. Java Fundamentals (Questions 1–20)

1. What is Java?

Answer: Java is a high-level, class-based, object-oriented programming language. Java source code is compiled into bytecode, which runs on a Java Virtual Machine (JVM).

2. What are the main features of Java?

Answer: Key features include platform independence, object-oriented programming, simplicity, security, robustness, portability, multithreading, and automatic memory management through garbage collection.

3. Why is Java called platform independent?

Answer: Java is platform independent because the compiler produces bytecode rather than machine code for one operating system. A compatible JVM can run that bytecode on different platforms. This is often summarized as “Write Once, Run Anywhere.”

4. What is the difference between source code, bytecode, and machine code?

Answer: Source code is the human-readable Java program (.java). Bytecode is the platform-neutral output of the Java compiler (.class). Machine code is processor-specific instructions executed directly by the CPU.

5. What is the JVM?

Answer: The Java Virtual Machine is the runtime environment that loads and executes Java bytecode. It also manages memory and supports features such as garbage collection and runtime checks.

6. What is the JDK?

Answer: The Java Development Kit is the software package used to develop Java programs. It includes tools such as the compiler (javac), launcher (java), debugger, and a runtime implementation.

7. What is the JRE?

Answer: The Java Runtime Environment traditionally means the components needed to run Java applications, including the JVM and supporting libraries. Modern Java distributions do not always provide a separate JRE download.

8. What is the difference between JDK, JRE, and JVM?

Answer: JVM executes bytecode. The JRE is the runtime components and libraries needed to run Java applications. The JDK includes development tools as well as the components needed to develop and run Java programs.

9. What is bytecode?

Answer: Bytecode is the intermediate instruction format produced by the Java compiler and stored in .class files. The JVM interprets or compiles it to native machine instructions.

10. Why is Java considered an object-oriented programming language?

Answer: Java organizes most program logic around classes and objects and supports encapsulation, inheritance, polymorphism, and abstraction. It also has primitive types, so not everything in Java is an object.

11. What is the role of the main() method in Java?

Answer: The main method is the conventional entry point the Java launcher calls to start a basic application. A common signature is public static void main(String[] args).

12. Why is main() declared as public static void?

Answer: public lets the launcher access it; static lets it run without creating an object; void means it returns no value. String[] args receives command-line arguments.

13. Can we execute a Java program without a main() method?

Answer: For a conventional standalone application, the launcher normally requires a valid main method. Some environments, frameworks, or newer Java launch modes may provide different entry mechanisms, but a beginner class-based program usually needs main().

14. What is a Java class?

Answer: A class is a blueprint that defines the data (fields) and behavior (methods) that objects of that type can have.

15. What is an object?

Answer: An object is a runtime instance of a class. It has state, represented by fields, and behavior, represented by methods.

16. What is the difference between a class and an object?

Answer: A class defines a type or blueprint; an object is a particular instance created from that class. For example, Car is a class and myCar can refer to one Car object.

17. What is a reference variable?

Answer: A reference variable stores a reference to an object, or null if it refers to no object. It does not contain the object's full data directly.

18. What is the difference between primitive data types and reference data types?

Answer: Primitive variables store values such as numbers, characters, or booleans. Reference variables refer to objects or arrays and can also hold null.

19. What are the eight primitive data types in Java?

Answer: byte, short, int, long, float, double, char, and boolean.

20. What is type casting?

Answer: Type casting converts a value from one data type to another. Widening conversions are often automatic, such as int to long; narrowing conversions, such as double to int, usually require an explicit cast and may lose information.

B. Variables, Operators and Control Statements (Questions 21–36)

21. What is a local variable?

Answer: A local variable is declared inside a method, constructor, or block. It is usable only within that scope and must be initialized before it is read.

22. What is an instance variable?

Answer: An instance variable is a non-static field declared in a class. Each object has its own copy, and it receives a default value if not explicitly initialized.

23. What is a static variable?

Answer: A static variable belongs to the class rather than to each individual object. There is one class-level variable shared by instances of that class.

24. What is the difference between == and =?

Answer: The = operator assigns a value. The == operator compares primitive values, or checks whether two reference variables refer to the same object.

25. What is the difference between == and .equals()?

Answer: For primitives, == compares values. For references, == compares identity (whether they refer to the same object); equals() compares logical equality when the class implements it appropriately, as String does.

26. What are relational operators in Java?

Answer: Relational operators compare values: <, >, <=, >=, ==, and !=. Their comparison result is a boolean.

27. What are logical operators?

Answer: Logical operators combine or invert boolean expressions. The main operators are && (conditional AND), || (conditional OR), and ! (NOT).

28. What is the difference between && and &?

Answer: For boolean expressions, && short-circuits: the right side is evaluated only if needed. & evaluates both sides. The & operator can also perform bitwise AND on integer values.

29. What is the ternary operator?

Answer: The ternary operator ?: is a compact conditional expression: condition ? valueIfTrue : valueIfFalse. Example: int max = (a > b) ? a : b;

30. What is the difference between break and continue?

Answer: break exits the nearest loop or switch. continue skips the rest of the current loop iteration and proceeds to the next iteration.

31. What is the difference between while and do-while?

Answer: while checks its condition before the body, so it may run zero times. do-while checks afterward, so its body runs at least once.

32. When would you use a for loop?

Answer: Use a for loop when repeating code with a clear initialization, condition, and update—often when the number of iterations is known or controlled by a counter.

33. What is a nested loop?

Answer: A nested loop is a loop inside another loop. For each iteration of the outer loop, the inner loop may run through its iterations.

34. What is a switch statement?

Answer: A switch selects one branch from several alternatives based on an expression's value. It can make multi-way selection clearer than a long if-else chain.

35. Can a switch statement work with a String?

Answer: Yes. Java supports String expressions in switch statements from Java 7 onward. Matching is case-sensitive.

36. What happens if break is omitted from a switch case?

Answer: In traditional switch statements, execution can fall through into the next case until a break or the end of the switch is reached. Modern switch expressions and arrow-style cases have different rules.

C. Arrays and Strings (Questions 37–55)

37. What is an array?

Answer: An array is an object that stores a fixed number of elements of one declared type. Elements are accessed by zero-based indexes.

38. Why are arrays called fixed-size data structures?

Answer: The length is set when the array is created and cannot be changed afterward. To hold more or fewer elements, create another array or use a resizable collection such as ArrayList.

39. How do you declare and initialize an array in Java?

Answer: Examples: int[] a = new int[3]; or int[] a = {10, 20, 30};. The first creates an array of three integers; the second creates and fills it with three values.

40. What is the default value of elements in an integer array?

Answer: Each int element defaults to 0. Other primitive arrays have their own defaults; reference-type array elements default to null.

41. How do you find the length of an array?

Answer: Use the length field, without parentheses: arrayName.length. For example, a.length.

42. What happens when you access an invalid array index?

Answer: Java throws ArrayIndexOutOfBoundsException at runtime when an index is negative or greater than or equal to the array length.

43. What is a two-dimensional array?

Answer: A two-dimensional array is an array whose elements are themselves arrays, often used like rows and columns. Example: int[][] matrix = new int[2][3];

44. What is the difference between an array and an ArrayList?

Answer: An array has a fixed length and can store primitives or references. ArrayList can grow or shrink and stores objects (primitive values are wrapped in types such as Integer). ArrayList provides collection methods such as add() and remove().

45. What is a String in Java?

Answer: String is a class representing a sequence of characters. String objects are immutable, meaning their contents cannot be changed after creation.

46. Why is String immutable?

Answer: Immutability improves safety and sharing, supports reliable hashing for uses such as map keys, and makes String objects easier to use across threads. Operations that appear to modify a String return a new String.

47. What is the difference between String, StringBuilder, and StringBuffer?

Answer: String is immutable. StringBuilder is mutable and usually preferred for repeated modifications in single-threaded code. StringBuffer is mutable and synchronizes its methods for thread safety, which can add overhead.

48. What is the difference between equals() and equalsIgnoreCase()?

Answer: equals() compares String contents with case sensitivity. equalsIgnoreCase() compares contents while ignoring differences in letter case.

49. What is the use of length() in a String?

Answer: String's length() method returns the number of UTF-16 code units in the string. Example: "Java".length() returns 4.

50. What is the use of substring()?

Answer: substring() returns part of a String. In substring(begin, end), begin is included and end is excluded; indexes are zero-based.

51. What is the use of charAt()?

Answer: charAt(index) returns the char at the specified zero-based index. An invalid index causes StringIndexOutOfBoundsException.

52. What is the difference between trim() and strip()?

Answer: Both remove whitespace from the beginning and end of a String. trim() uses a legacy, limited character definition; strip(), available from Java 11, uses Unicode-aware whitespace rules.

53. What do toUpperCase() and toLowerCase() do?

Answer: They return a String converted to uppercase or lowercase, respectively. The original String remains unchanged because String is immutable.

54. What is the use of contains() in String?

Answer: contains() checks whether a String contains the specified character sequence and returns true or false. Example: "Java Programming".contains("Java") returns true.

55. What is the difference between concat() and the + operator for Strings?

Answer: Both can join strings and produce a result String. + is more flexible and can combine strings with other values; concat() takes a String argument. Neither changes the original String.

D. OOP Concepts (Questions 56–80)

56. What are the four main principles of OOP?

Answer: Encapsulation, inheritance, polymorphism, and abstraction.

57. What is encapsulation?

Answer: Encapsulation combines data and the methods that operate on it in a class, while controlling access to the internal state.

58. How is encapsulation achieved in Java?

Answer: Commonly, declare fields private and provide public methods such as getters and setters to read or update them with validation when needed.

59. What is inheritance?

Answer: Inheritance lets a class derive fields and methods from another class. The child class uses extends to inherit from its parent class.

60. Why is inheritance used?

Answer: It supports code reuse and allows specialized classes to extend or customize existing behavior. It also enables runtime polymorphism through overridden methods.

61. What is the difference between extends and implements?

Answer: extends establishes class inheritance or interface inheritance. implements declares that a class provides the behavior required by one or more interfaces.

62. What is polymorphism?

Answer: Polymorphism means one interface or parent type can represent objects of different concrete types, allowing the appropriate implementation to be selected. Overloading is compile-time polymorphism; overriding enables runtime polymorphism.

63. What is compile-time polymorphism?

Answer: It is commonly achieved through method overloading, where the compiler chooses a matching method based on the method signature and argument types.

64. What is runtime polymorphism?

Answer: It occurs when an overridden instance method is selected according to the actual object's class at runtime, even if the reference uses a parent type.

65. What is method overloading?

Answer: Method overloading means defining methods with the same name but different parameter lists (different parameter types, number, or order) in the same class or inheritance context.

66. What is method overriding?

Answer: Method overriding occurs when a subclass provides a compatible implementation of an inherited instance method, using the same signature and a compatible return type.

67. What is the difference between method overloading and overriding?

Answer: Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding replaces an inherited instance-method implementation and is dispatched at runtime.

68. Can we overload a method by changing only its return type?

Answer: No. The parameter list must differ. Changing only the return type makes the method signature ambiguous for overload resolution.

69. What is the super keyword?

Answer: super refers to the immediate superclass. It can call superclass methods, access accessible superclass members, or invoke a superclass constructor using super(...).

70. What is the this keyword?

Answer: this refers to the current object. It can distinguish fields from parameters (this.name = name), call another constructor in the same class using this(...), or pass the current object as an argument.

71. What is a constructor?

Answer: A constructor is a special class member used to initialize a new object. It has the class name and no return type.

72. What is the difference between a constructor and a method?

Answer: A constructor initializes objects, has the class name, and has no return type; it is invoked during object creation. A method defines behavior, has a return type (or void), and is called explicitly or through program flow.

73. Can a constructor be overloaded?

Answer: Yes. A class can have multiple constructors with different parameter lists, allowing objects to be initialized in different ways.

74. Can a constructor be inherited?

Answer: No. Constructors are not inherited by subclasses. A subclass constructor can call a superclass constructor using super(...).

75. What is an abstract class?

Answer: An abstract class is declared with abstract and cannot be instantiated directly. It may contain abstract methods without bodies, as well as implemented methods, fields, and constructors.

76. What is an interface?

Answer: An interface defines a contract that implementing classes agree to follow. It can declare abstract methods and may also contain default, static, and private methods, plus constants.

77. Why do we use interfaces?

Answer: Interfaces define common capabilities without requiring classes to share a parent class. They support loose coupling, abstraction, and multiple-interface implementation.

78. What is the difference between an abstract class and an interface?

Answer: An abstract class can hold instance state, constructors, and both abstract and concrete methods; a class can extend only one class. A class can implement multiple interfaces, which primarily define contracts and can also contain default and static behavior.

79. Can a class implement multiple interfaces?

Answer: Yes. A class can implement more than one interface, separating the capabilities it promises to provide. Example: class Demo implements Runnable, AutoCloseable.

80. Can an interface extend another interface?

Answer: Yes. An interface can extend one or more interfaces using extends, inheriting their contracts.

E. Access Modifiers, Packages & Core Concepts (Questions 81–90)

81. What are the access modifiers in Java?

Answer: The four access levels are private, default (package-private, when no modifier is written), protected, and public.

82. What is the difference between private, default, protected, and public?

Answer: private: accessible within the declaring class. Default/package-private: accessible within the same package. protected: accessible within the same package and in subclasses subject to Java's access rules. public: accessible wherever the class/member is otherwise accessible.

83. Why are data members generally declared private?

Answer: Private fields protect an object's internal state and prevent uncontrolled direct changes. Public methods can validate values and preserve class rules.

84. What is a package?

Answer: A package is a namespace that groups related Java types, helps organize code, and controls package-level access. Example: java.util.

85. Why are packages used in Java?

Answer: Packages organize related classes, avoid naming conflicts, support access control, and make large projects easier to maintain.

86. What is the difference between import and package?

Answer: package declares which package a source file's type belongs to. import lets code refer to types from other packages by their simple names instead of fully qualified names.

87. What is the purpose of the static keyword?

Answer: static makes a field or method belong to the class rather than a particular instance. Static methods can be called using the class name and cannot directly access instance members without an object.

88. What is the purpose of the final keyword?

Answer: final prevents reassignment of a variable, overriding of a method, or inheritance from a class. For a reference variable, final prevents changing the reference, not necessarily modifying the referenced object.

89. Can a final variable be modified?

Answer: A final variable cannot be reassigned after it has been initialized. If it refers to a mutable object, the object's contents may still be changed.

90. Can a final method be overridden?

Answer: No. A final method cannot be overridden by a subclass, although it can be overloaded with a different parameter list.

F. Exception Handling (Questions 91–100)

91. What is an exception?

Answer: An exception is an event represented by an object that disrupts the normal flow of program execution. Java uses classes derived from Throwable to represent exceptions and errors.

92. What is exception handling?

Answer: Exception handling is the process of detecting and responding to exceptional conditions using constructs such as try, catch, finally, throw, and throws.

93. What is the difference between an error and an exception?

Answer: Both are subclasses of Throwable. Exceptions are conditions applications may often handle; Errors generally indicate serious JVM or system problems that applications usually should not try to recover from.

94. What is the difference between checked and unchecked exceptions?

Answer: Checked exceptions must be caught or declared with throws, as enforced by the compiler. Unchecked exceptions—RuntimeException and its subclasses—do not have that requirement. Errors are also unchecked.

95. What is the purpose of try and catch?

Answer: Code that may throw an exception is placed in try. A matching catch block handles the exception so the program can respond instead of immediately propagating it.

96. What is the purpose of the finally block?

Answer: finally normally runs after try/catch whether an exception occurs or not, making it useful for cleanup. It may not run if the JVM terminates abruptly; try-with-resources is often preferred for closing resources.

97. What is the difference between throw and throws?

Answer: throw is used inside code to explicitly throw a particular exception object. throws appears in a method or constructor declaration to list exceptions that may propagate to the caller.

98. Can we create our own exception class?

Answer: Yes. Define a class that extends Exception for a checked exception or RuntimeException for an unchecked exception. Example: class InvalidAgeException extends Exception {}.

99. What happens if an exception is not handled?

Answer: If it is not caught in the current call path, it propagates up the call stack. If it remains uncaught, the thread's uncaught-exception handler is invoked and that thread typically terminates after a stack trace is printed.

100. Why should exceptions be handled instead of allowing the program to terminate unexpectedly?

Answer: Appropriate handling can provide a useful message, recover when possible, clean up resources, and keep the application in a consistent state. Exceptions should be handled meaningfully rather than hidden with empty catch blocks.